# PYQS

## Ch-1

1) Which of the OSI layers handles each of the following:
   i) Dividing the message into segments.
   ii) Determining which route through the subnet to use.
   iii) Dividing the transmitted bit stream into frames.

**i) Dividing the message into segments.**

- **Answer**: **Transport Layer (Layer 4)**.
  - The **Transport Layer** is responsible for dividing larger messages into smaller **segments** for more efficient transmission. It ensures error control, flow control, and reliable communication between hosts.

**ii) Determining which route through the subnet to use.**

- **Answer**: **Network Layer (Layer 3)**.
  - The **Network Layer** is in charge of determining the optimal route for data packets through the network or subnet. It manages packet forwarding and routing based on IP addresses.

**iii) Dividing the transmitted bit stream into frames.**

- **Answer**: **Data Link Layer (Layer 2)**.
  - The **Data Link Layer** is responsible for dividing the raw bit stream into **frames** and handles error detection and correction at this level. It ensures that frames are transmitted correctly between devices on the same local network.

2) Besides bandwidth and latency, what other parameter(s) is/are needed togive a good characterization of the quality of service offered by network used for
   (i) Online financial transaction traffic?
   (ii) Video streaming traffic?

**(i) Online Financial Transaction Traffic:**

- **Reliability**: Financial transactions must be delivered accurately and without loss or corruption. Mechanisms like acknowledgment and error correction are crucial.
- **Security**: Ensuring that sensitive data is encrypted and protected from unauthorized access is critical in financial transactions.

**(ii) Video Streaming Traffic:**

- **Packet Loss**: Some packet loss can degrade video quality, so minimizing packet loss is important.
- **Throughput**: High throughput is needed to support continuous video data transfer, especially for high-resolution streams.

3) Discuss the circuit switching versus packet switching approaches for moving data through a network of links and switches.

## Circuit Switching:

**Overview**:
In circuit switching, a dedicated communication path or circuit is established between the sender and receiver before any data transmission begins. This path remains reserved and exclusive for the duration of the session.

**Key Characteristics**:

- **Connection-Oriented**: A physical path (or virtual circuit) must be established before data transfer begins. This connection remains active until the communication is completed.
- **Dedicated Resources**: Resources (bandwidth, switches, etc.) are reserved for the entire duration of the connection, whether or not data is being transmitted.
- **Fixed Bandwidth**: The bandwidth allocated remains constant throughout the communication session.
- **Delay**: There is an initial delay in setting up the circuit. Once established, communication has minimal latency, as data flows continuously along the dedicated path.

**Examples**:

- **Traditional Telephone Networks**: In a phone call, a dedicated circuit is reserved between the caller and the receiver.

**Advantages**:

- **Predictable Performance**: Since resources are dedicated, the quality of service (QoS) is predictable, with no contention for bandwidth.
- **No Congestion**: No packet loss or network congestion, as the connection is exclusive.

**Disadvantages**:

- **Inefficient Use of Resources**: Resources are reserved for the entire duration, even if there is no data transmission (e.g., during silence in a phone call).
- **Scalability**: Difficult to scale for large networks because it requires dedicated resources for each session.

## Packet Switching:

**Overview**:
In packet switching, data is divided into small, independent packets that are sent through the network. Each packet may take different routes to the destination, and no dedicated path is required.

**Key Characteristics**:

- **Connectionless (or Connection-Oriented in some cases)**: Packets are routed independently through the network. There is no need for a dedicated connection, although some protocols (like TCP) establish logical connections.
- **Dynamic Routing**: Each packet can take a different path based on current network conditions, such as congestion and link availability.
- **Efficient Resource Utilization**: Network resources are shared dynamically among many users. Bandwidth is used as needed, making packet switching more efficient, especially in bursty traffic scenarios.
- **Variable Delay (Latency)**: Since packets may take different routes and face varying levels of congestion, delays can vary, which may affect real-time applications like video or voice.

**Examples**:

- **Internet Traffic**: Most internet protocols, including HTTP (web browsing) and email, use packet switching.

**Advantages**:

- **Efficient Bandwidth Utilization**: Resources are shared and used only when needed, optimizing the network's capacity.
- **Scalability**: Easy to scale to large networks, like the Internet, where multiple users share the same infrastructure.
- **Fault Tolerance**: If a particular route is down or congested, packets can be rerouted dynamically.

**Disadvantages**:

- **Variable Performance**: Network performance can fluctuate due to congestion or routing decisions. Latency, jitter, and packet loss can occur.
- **Overhead**: Each packet requires header information, which introduces overhead and reduces efficiency for small data transfers.

| Aspect | Circuit Switching | Packet Switching |
|---|---|---|
| **Connection** | Dedicated path (connection-oriented) | No dedicated path (connectionless or logical connection) |
| **Resource Allocation** | Resources are reserved for the entire session | Resources are shared dynamically |
| **Efficiency** | Inefficient, as resources are reserved even during inactivity | Efficient, as resources are used only when needed |
| **Delay** | Low latency after setup, but circuit setup takes time | Variable latency, depending on routing and network congestion |
| **Use Case** | Voice calls, traditional telephony networks | Data transfer (Internet, email, web browsing, video streaming) |
| **Scalability** | Limited scalability, requires dedicated paths | Highly scalable, dynamic routing supports larger networks |

## 4) Draw and explain different layers of OSI Model.

From notes and Darshan

## 5) Find transmission delay for transmission of 1500 bytes (12000 bits) using a transmission rate of 100Mbps.

### 5) Transmission Delay Calculation:

**Transmission Delay** is the time required to push all the bits of a data packet onto the transmission medium. It can be calculated using the formula:

$$\text{Transmission Delay} = \frac{\text{Size of Data (in bits)}}{\text{Transmission Rate (in bits per second)}}$$

**Given Data:**

- Size of data: 1500 bytes = 12000 bits
- Transmission rate: 100 Mbps (Megabits per second) = $100 \times 10^6$ bits per second

Now, using the formula:

$$\text{Transmission Delay} = \frac{12000 \text{ bits}}{100 \times 10^6 \text{ bits per second}} = 0.00012 \text{ seconds} = 120 \text{ microseconds}$$

So, the transmission delay is **120 microseconds**.

## 6) What is the functionality of Router, Bridge and Gateway?

**Router:**

- **Function**: A router is a networking device that forwards data packets between different networks. It operates at the **Network Layer (Layer 3)** of the OSI model.
- **Key Features**:
  - Routes data based on **IP addresses**.
  - Connects **different networks**, such as connecting a home network to the internet.
  - Uses routing tables and algorithms to determine the optimal path for data.
  - Provides **NAT (Network Address Translation)**, allowing multiple devices on a private network to share a single public IP.

    - Supports **firewall functionality** to filter incoming and outgoing traffic.

**Bridge:**

- **Function**: A bridge is a device that connects and filters traffic between two or more segments of a **local area network (LAN)**. It operates at the **Data Link Layer (Layer 2)** of the OSI model.
- **Key Features**:
    - Connects different network segments within the same LAN to form a **single, larger network**.
    - Filters traffic by looking at **MAC addresses** to decide whether to forward or discard a frame.
    - Helps reduce network traffic by dividing a network into segments, improving performance and reducing collisions in large LANs.

**Gateway:**

- **Function**: A gateway is a device that connects networks using **different protocols**. It operates at various layers of the OSI model, typically from the **Transport Layer (Layer 4)** and higher.
- **Key Features**:
    - Acts as a translator between different network **protocols** (e.g., connecting an IP network to a legacy protocol network).
    - Provides access between a **private network** and external networks (often the internet).
    - Can be used in various functions such as **VoIP**, email translation, and protocol conversion.
    - Typically performs more complex functions than routers and bridges, often including security features.

In summary:

- A **router** forwards data between different networks based on IP addresses.
- A **bridge** connects segments of the same network and filters traffic based on MAC addresses.
- A **gateway** serves as a translator between networks using different protocols.

# Ch-2

## 1) Justify the statement, “HTTP server is stateless”.

The statement "**HTTP server is stateless**" refers to the fact that the **HTTP protocol** (HyperText Transfer Protocol), which web servers use to communicate with clients (typically web browsers), does not retain information about previous client requests between connections.

### Explanation and Justification:

1. **No Memory of Previous Requests**:
    - In an **HTTP server**, each request from a client (such as a browser) is treated as an independent, isolated transaction. The server processes each request without any knowledge of the previous interactions with that same client.
    - For example, if you request one webpage, and then immediately request another, the server has no memory of your first request while processing the second one. Each request is handled as though it were from a completely new user.
2. **Stateless Protocol**:
    - A **stateless protocol** is one in which the server does not store any information or session data about the client after the transaction is completed. HTTP is designed this way to make the protocol simpler and more scalable, as the server doesn’t need to keep track of every client.
    - Once the server responds to an HTTP request, the connection is closed, and the server does not retain any data about the client, the request, or the respons

An **HTTP server is stateless** because it does not inherently remember any information about clients or previous requests. Each interaction between the client and server is independent, and any stateful behavior must be managed by other techniques (e.g., cookies or sessions) to persist data across multiple requests. This stateless nature of HTTP makes it simpler, more scalable, and more flexible for the web’s widespread use.

## 2) State the port number for the following application layer protocols.

1) i) FTP ii) HTTP iii) SMTP iv) POP3

**FTP (File Transfer Protocol)**:

- Port 20 (Data Transfer)
- Port 21 (Control Commands)

**HTTP (HyperText Transfer Protocol)**:

- Port 80

**SMTP (Simple Mail Transfer Protocol)**:

- Port 25

**POP3 (Post Office Protocol version 3)**:

- Port 110

## 3) Discuss the five layer internet protocol stack along with the functionalities of each layer in detail. (Ch-1)

The **Five-Layer Internet Protocol Stack** (also known as the TCP/IP model) is a conceptual framework used to understand and design network communications. It defines how data is transmitted from one device to another over a network. Each layer has distinct functionalities and interacts with the layers directly above and below it.

### The Five Layers of the Internet Protocol Stack:

### 1. Physical Layer (Layer 1)

- **Functionality**:
  - The **Physical Layer** is responsible for the **transmission of raw bit streams** over a physical medium, such as cables (copper, fiber optics) or wireless signals.
  - It defines the electrical, mechanical, and procedural standards for the hardware that transmits and receives data.
  - It specifies the **hardware** characteristics such as voltage levels, timing of signals, data rates, physical connectors, and transmission modes (simplex, half-duplex, full-duplex).
- **Examples**:
  - Ethernet cables, optical fibers, wireless radio frequencies.

## 2. Data Link Layer (Layer 2)

- **Functionality**:
  - The **Data Link Layer** is responsible for **node-to-node communication** and error detection between devices connected on the same network (i.e., local area network or LAN).
  - It handles the division of the data stream into **frames**, which are smaller data units.
  - This layer ensures **error detection** and correction, and manages **flow control** to prevent data overloading.
  - It uses **MAC (Media Access Control) addresses** to identify devices within the same network and decide how data is forwarded to devices within the local network.
- **Sub-layers**:
  - **Logical Link Control (LLC)**: Controls frame synchronization, flow control, and error checking.
  - **Media Access Control (MAC)**: Controls how devices on a network gain access to data and permission to transmit it.
- **Examples**:
  - Ethernet, Wi-Fi, MAC addresses, switches.

## 3. Network Layer (Layer 3)

- **Functionality**:
  - The **Network Layer** is responsible for **routing** data between different networks and ensuring that packets reach their destination across multiple networks (i.e., internetworking).
  - It defines **logical addressing** (such as **IP addresses**) and **path determination** to forward packets to their destination.
  - It handles **packet forwarding**, **routing**, and **fragmentation** of large packets when necessary.
  - Routers, which are responsible for routing packets based on IP addresses, operate at this layer.
- **Examples**:
  - IP (Internet Protocol), ICMP (Internet Control Message Protocol), Routers.

## 4. Transport Layer (Layer 4)

- **Functionality**:
  - The **Transport Layer** ensures **end-to-end communication** between devices, providing reliable data transfer and error recovery.
  - It establishes, maintains, and terminates connections between applications.
  - This layer is responsible for **segmentation** of data (dividing larger messages into smaller segments) and **reassembly** at the destination.
  - It provides **flow control**, **error detection**, and **retransmission** in case of errors or data loss.
  - Two main protocols operate here:
    - **TCP (Transmission Control Protocol)**: Ensures reliable, ordered, and error-checked delivery of data (connection-oriented).
    - **UDP (User Datagram Protocol)**: Provides a faster but less reliable service (connectionless).
- **Examples**:
  - TCP (Transmission Control Protocol), UDP (User Datagram Protocol).

## 5. Application Layer (Layer 5)

- **Functionality**:
  - The **Application Layer** is the topmost layer that directly interacts with the **end-user**.
  - It provides services to applications, allowing them to communicate over a network. It defines how applications like web browsers, email clients, and file transfer utilities interact with the underlying network.
  - This layer encompasses high-level protocols that manage things like **file transfers**, **email communication**, and **web browsing**.
  - The Application Layer abstracts the complexity of lower layers and provides a simple interface for the user.
- **Examples**:
  - HTTP (HyperText Transfer Protocol), FTP (File Transfer Protocol), SMTP (Simple Mail Transfer Protocol), DNS (Domain Name System).

## Summary of Functions of Each Layer:

1. **Physical Layer**: Transmits raw bits over a physical medium.

2. **Data Link Layer**: Ensures reliable communication between nodes in a local network (frames, MAC addresses).
3. **Network Layer**: Routes data across different networks (packets, IP addressing).
4. **Transport Layer**: Provides end-to-end communication, ensuring reliable data transfer (TCP/UDP).
5. **Application Layer**: Interacts with end-user applications and provides services like web browsing, file transfer, and email (HTTP, FTP, SMTP).

## 4) Explain HTTP and DNS in detail.

**HTTP (HyperText Transfer Protocol)** is the foundational protocol used for transmitting data over the web. It defines the rules and standards that govern how web browsers (clients) and web servers communicate. HTTP operates at the **Application Layer** of the Internet protocol stack and follows a request-response model.

**Key Characteristics of HTTP:**

1. **Stateless**:
   - HTTP is **stateless**, meaning that each request from a client is treated as an independent transaction. The server does not retain any information about previous requests, although mechanisms like cookies and sessions can be used to maintain state.
2. **Request-Response Model**:
   - HTTP communication happens through a client (typically a web browser) sending a request to the server, and the server responds with the requested resource (such as a web page, image, or other content).
   - The communication flow looks like:
     - **Client Request**: The client sends an HTTP request (e.g., GET or POST) to the server for a resource (like a web page).
     - **Server Response**: The server processes the request and sends back an HTTP response, often containing the requested resource (e.g., HTML content, an image, etc.).

**HTTP Methods:**

HTTP uses different methods (also called **verbs**) to indicate the type of action being requested by the client.

- **GET**: Used to request data from the server (e.g., accessing a webpage). It retrieves a resource without modifying any server-side data.
- **POST**: Sends data to the server (e.g., submitting a form). The data is often used to create or update a resource.
- **PUT**: Replaces all current representations of the target resource with the uploaded content.
- **DELETE**: Removes a specified resource from the server.
- **HEAD**: Similar to GET, but it only retrieves the headers, not the actual content of the resource.
- **OPTIONS**: Describes the communication options for the target resource.

**HTTP Response Status Codes:**

After the server processes a request, it returns a response that includes a **status code** to indicate the outcome of the request.

- **200 OK**: The request was successful, and the requested resource is sent back to the client.
- **404 Not Found**: The requested resource could not be found on the server.
- **500 Internal Server Error**: There was an error on the server, preventing it from fulfilling the request.
- **301 Moved Permanently**: The resource has been moved to a new URL.

The **Domain Name System (DNS)** is a hierarchical and decentralized system used to resolve human-readable domain names (like www.example.com) into IP addresses (such as 192.168.1.1), which are required for devices to communicate over the Internet. DNS is essentially the **phonebook of the Internet**, mapping names to IP addresses.

**Key Functions of DNS:**

1. **Name Resolution**:
    - The primary function of DNS is to translate domain names into **IP addresses**. This process is called **name resolution**.
    - When you type a domain name in a web browser, the DNS system looks up the corresponding IP address so that your browser can communicate with the server hosting the website.
2. **Hierarchical Structure**:

- DNS uses a hierarchical structure with different levels, organized as a tree with the **root domain** at the top.
- Example domain hierarchy for "www.example.com":
  - **Root Domain**: Represented by a dot (`.`) at the top.
  - **Top-Level Domain (TLD)**: The highest-level domain, such as **.com**, **.org**, **.net**.
  - **Second-Level Domain**: The part directly to the left of the TLD (in this case, **example**).
  - **Subdomains**: Prefixes added to a domain name (like **www** in www.example.com).

3. **Distributed System**:
   - DNS is a **distributed database**. Rather than being a single database managed by one entity, DNS is distributed across many servers worldwide. Each server holds information for specific parts of the DNS hierarchy.
4. **Caching**:
   - DNS employs caching to reduce lookup times and network traffic. When a DNS server resolves a domain name, it temporarily stores the result. Future requests for the same domain can be answered from the cache, reducing the need for further lookups.

HTTP enables web communication, while DNS ensures users can find websites using familiar domain names instead of complex IP addresses. Both are critical components of the modern Internet.

# Ch-3

1) Explain User Datagram Protocol (UDP) in detail and discuss how it differs from Transmission Control Protocol (TCP).
2) Compare TCP and UDP.

**UDP (User Datagram Protocol)** is a **connectionless**, simple, and lightweight communication protocol that operates at the **Transport Layer** of the Internet Protocol (IP) suite. It is used for applications that require speed and efficiency, where reliability and error correction are less critical.

**Key Characteristics of UDP:**

1. **Connectionless Protocol**:
   - UDP is **connectionless**, meaning it doesn't establish a connection before transmitting data. It sends packets called **datagrams** directly to the recipient without any prior handshaking process.
   - There's no formal setup phase, unlike in TCP, which requires a three-way handshake to establish a connection.
2. **No Guarantee of Delivery**:
   - UDP does not provide mechanisms to ensure that data reaches its destination. There's no **error recovery**, **retransmission**, or **acknowledgment** of receipt.
   - If a packet is lost or arrives out of order, UDP will not attempt to correct it.
3. **No Error Checking or Flow Control**:
   - UDP provides **minimal error checking** using a simple checksum but does not include robust mechanisms for **error correction** or **flow control**.
   - The protocol is designed to be fast, sacrificing reliability in favor of lower latency and higher throughput.
4. **Lightweight**:
   - UDP has a smaller **header size** compared to TCP, which reduces overhead and allows for quicker data transmission. The UDP header is only 8 bytes long, compared to TCP's 20-byte header.
5. **Low Latency**:
   - Since there's no connection establishment, acknowledgment, or error correction, UDP has **lower latency** compared to TCP, making it suitable for real-time applications like gaming, video streaming, and VoIP.

**UDP Header Structure:**

The UDP header is simple and consists of four fields, each 16 bits (2 bytes) long:

1. **Source Port**: The port number of the sending application.
2. **Destination Port**: The port number of the receiving application.
3. **Length**: The length of the UDP header and data.
4. **Checksum**: Used for error detection, but errors are not corrected if detected.

| Aspect | UDP (User Datagram Protocol) | TCP (Transmission Control Protocol) |
|---|---|---|
| **Connection** | Connectionless, no need to establish a connection before sending data. | Connection-oriented, requires a three-way handshake before transmission. |
| **Reliability** | Unreliable: no acknowledgment, no retransmission, no guarantee of data delivery. | Reliable: guarantees delivery through acknowledgments, retransmission of lost packets, and error correction. |
| **Data Transmission** | Sends data in **datagrams** without confirming receipt or ensuring proper sequencing. | Sends data in **segments** with acknowledgment, sequencing, and flow control. |
| **Error Detection/Correction** | Minimal error detection using a checksum but no error correction. | Comprehensive error detection, correction, and recovery. If errors are detected, TCP retransmits packets. |
| **Flow Control** | No flow control, which means it cannot adjust to the speed of the receiver. | Flow control using the **Sliding Window** mechanism, ensuring that the sender does not overwhelm the receiver. |
| **Congestion Control** | No congestion control, it continues sending data regardless of network conditions. | Implements congestion control algorithms (e.g., TCP Reno) to avoid overwhelming the network. |
| **Overhead** | Lightweight, with a small 8-byte header, leading to faster transmission. | More overhead due to a larger header (20 bytes or more) and the need for handshakes and acknowledgments. |
| **Transmission Speed** | Faster due to minimal overhead and no need for handshakes or acknowledgments. | Slower because of the overhead involved in establishing connections, error-checking, and acknowledgments. |
| **Use Cases** | Suitable for real-time applications where speed is critical, and some data loss is tolerable (e.g., video streaming, VoIP, gaming). | Used for applications requiring reliable data transmission, such as file transfers (FTP), emails (SMTP), and web browsing (HTTP). |
| **Data Ordering** | No guarantee of order: packets may arrive out of sequence. | Guarantees in-order delivery of packets through sequence numbers. |
| **Connection Termination** | No formal connection, so no termination is needed. | Requires proper connection termination using a **four-way handshake**. |

## When to Use UDP:

- **Real-time applications** where small delays are more important than data reliability.
- **Applications where minor data loss is acceptable**, such as streaming or online games.
- **Broadcast or multicast** transmission (sending data to multiple recipients simultaneously).

## When to Use TCP:

- **Applications that require guaranteed delivery** of data and error recovery, such as file transfers or email.
- **Situations where the data must be sent in sequence** and without loss (e.g., downloading a file, accessing a website)

## 2) For the below mentioned internet applications protocol, mention the underlying transport protocol (TCP or UDP).

## i) Telnet ii) FTP iii) HTTP

**Telnet**: **TCP**

- Telnet is a network protocol used for remote login and command execution on devices over a TCP/IP network. It uses TCP for reliable communication.

**FTP (File Transfer Protocol)**: **TCP**

- FTP is used for transferring files between a client and server on a network. It operates over TCP to ensure reliable and ordered delivery of data.

**HTTP (HyperText Transfer Protocol)**: **TCP**

- HTTP is the foundation of data communication on the World Wide Web. It relies on TCP for reliable transmission of web resources such as HTML pages, images, and other content.

## 3) Explain the UDP checksum mechanism for error detection with example.

—

4) Differentiate between connection oriented and connection less service.

| Feature | Connection-Oriented Service | Connectionless Service |
|---|---|---|
| **Connection Establishment** | Requires a connection establishment phase (e.g., TCP three-way handshake). | No connection establishment; data is sent without prior setup. |
| **Reliability** | Reliable; guarantees data delivery and correct ordering. | Unreliable; does not guarantee delivery or ordering. |
| **Data Flow Control** | Provides flow control mechanisms to manage data transmission rates. | No flow control; each packet is treated independently. |
| **Overhead** | Higher overhead due to connection management (e.g., handshakes). | Lower overhead as no connection setup is needed. |
| **Use Cases** | Suitable for applications where reliability is crucial (e.g., file transfers, emails). | Suitable for applications where speed is essential and some data loss is acceptable (e.g., streaming, gaming). |
| **Example Protocols** | TCP (Transmission Control Protocol). | UDP (User Datagram Protocol). |
| **Session Termination** | Requires proper termination procedures (e.g., TCP four-way handshake). | No formal termination; data is sent independently. |

## 5) Compare Datagram and virtual circuit.

| Feature | Datagram | Virtual Circuit |
|---|---|---|
| **Connection Type** | Connectionless; each packet is sent independently without prior setup. | Connection-oriented; a logical connection is established before data transmission. |
| **Reliability** | Unreliable; no guarantees on delivery, order, or error correction. | Reliable; guarantees data delivery and order, often includes error correction. |
| **Data Flow Control** | No flow control; packets may arrive out of order. | Flow control mechanisms are used to manage data transmission rates. |
| **Overhead** | Lower overhead as there's no connection setup. | Higher overhead due to connection management. |
| **Routing** | Each datagram can take a different route to the destination. | All packets follow the same predetermined path through the network. |
| **Use Cases** | Suitable for applications requiring low latency and tolerating some data loss (e.g., VoIP, streaming). | Suitable for applications requiring reliable and ordered delivery (e.g., video conferencing, online gaming). |
| **Example Protocols** | IP (Internet Protocol). | X.25, ATM (Asynchronous Transfer Mode). |

## 6) What are the services provided to transport layer by the network layer?

The transport layer relies on the services provided by the network layer to facilitate communication between devices over a network. Here are the primary services that the network layer provides to the transport layer:

### 1. Packet Forwarding

- The network layer is responsible for forwarding packets from the source to the destination across multiple networks or segments. It determines the best path for data transmission.

### 2. Logical Addressing

- The network layer assigns logical addresses (e.g., IP addresses) to devices, which allows for identification and routing of packets to the correct destination. This is crucial for the transport layer to identify endpoints correctly.

### 3. Routing

- The network layer handles routing decisions, determining the best route for packets to travel through the network. This involves using routing algorithms and maintaining routing tables.

### Error Handling

- While the transport layer often handles error detection and recovery, the network layer can also contribute by detecting and reporting some types of errors during packet forwarding. For example, it can detect unreachable destinations and inform the transport layer.

### Network Layer Protocols

- The network layer operates with protocols such as Internet Protocol (IP), which the transport layer utilizes for delivering data. The transport layer protocols (like TCP and UDP) depend on the underlying network layer protocols for routing and addressing.

# Ch-4

## 1) Discuss the count-to-infinity problem in distance vector routing algorithm with example.

The **count-to-infinity problem** is a well-known issue in distance vector routing protocols, such as RIP (Routing Information Protocol), where routers may take a long time to converge to a stable routing table after a change in the network topology. This problem arises due to the nature of the distance vector algorithm, which relies on the exchange of routing information between directly connected neighbors.

**Explanation of the Problem**

1. **Distance Vector Routing**:
   - In a distance vector routing protocol, each router maintains a table (the distance vector) that holds the best-known distance (cost) to reach each destination and the next hop to reach that destination.
   - Routers periodically exchange their distance vectors with neighbors to update their routing tables.
2. **Count-to-Infinity**:
   - The count-to-infinity problem occurs when there is a network topology change, such as a link failure, causing routers to update their distance vectors.
   - If a router cannot reach a destination due to a link failure, it will increment the distance to that destination in its routing table.
   - The increment continues until the distance metric reaches the maximum limit (infinity). This process can take a long time as the routers continuously propagate the incorrect distance to each other.

### Preventing the Count-to-Infinity Problem

Several techniques can help mitigate the count-to-infinity problem:

1. **Split Horizon**: A router does not advertise a route back to the neighbor from which it learned it.
2. **Route Poisoning**: If a route becomes unreachable, a router immediately advertises it as having an infinite distance (e.g., metric of 16 in RIP).
3. **Hold-down Timers**: Temporarily prevent updates for a certain period, allowing the network to stabilize before new routing information is accepted

## 2) Explain Link State Routing algorithm in detail.

In contrast to distance vector routing, link state routing allows routers to have a complete view of the network topology, enabling more efficient and faster routing decisions.

**Key Characteristics of Link State Routing**

1. **Complete Network Topology**:
   - Each router maintains a complete map of the network topology, represented as a graph where nodes are routers and edges are the links between them.
2. **Link State Advertisements (LSAs)**:
   - Routers send out LSA packets that contain information about their directly connected neighbors and the state of those links (cost, bandwidth, etc.).
3. **Dijkstra's Algorithm**:
   - Link state protocols typically use Dijkstra's algorithm to calculate the shortest path to each destination based on the complete topology.
4. **Fast Convergence**:
   - Link state protocols generally converge faster than distance vector protocols because changes in the topology are propagated quickly and allow routers to independently calculate the best routes.

**Advantages of Link State Routing**

- **Faster Convergence**: Changes in the network are quickly propagated, leading to faster convergence.
- **Loop-Free Paths**: The algorithm inherently avoids routing loops due to the use of complete topology information.
- **Scalability**: Better suited for larger and more complex networks compared to distance vector protocols.

**Disadvantages of Link State Routing**

- **Memory and CPU Intensive**: Requires more memory and processing power to store and compute the complete network topology.
- **Complexity**: More complex than distance vector protocols, requiring more sophisticated algorithms.

# 3) Explain distance vector routing with suitable example.

**Distance Vector Routing** is a routing protocol used in networks that allows routers to share information about the best paths to reach various destinations based on the distance metric (e.g., hops, cost). Each router maintains a routing table (distance vector) that includes the best-known distance to each destination and the next hop to reach that destination.

## Key Characteristics of Distance Vector Routing

1. **Distance Metric**: Each router calculates the best path to each destination using a distance metric, such as the number of hops or the cost associated with the path.

2. **Periodic Updates**: Routers periodically send their entire routing tables to their directly connected neighbors. This helps keep routing information synchronized across the network.

3. **Bellman-Ford Algorithm**: The algorithm used to compute the best paths based on the distance vectors received from neighbors.

4. **Routing Loops**: Distance vector routing is prone to routing loops, which can occur when there are inconsistencies in routing information. Techniques such as split horizon and route poisoning are used to mitigate this issue.

## Example of Distance Vector Routing

Let's consider a simple network with three routers: **A**, **B**, and **C**. The initial distances to reach each router are as follows:

- A to B: Cost = 1 (directly connected)
- A to C: Cost = 4 (directly connected)
- B to C: Cost = 1 (directly connected)

**Initial Distance Vector Tables**

At the start, each router has the following distance vector tables:

| Router | Destination | Cost | Next Hop |
|---|---|---|---|
| A | B | 1 | B |
| A | C | 4 | C |
| B | A | 1 | A |
| B | C | 1 | C |
| C | A | ∞ | - |
| C | B | 1 | B |

- **Router A** knows it can reach Router B at a cost of 1 and Router C at a cost of 4.
- **Router B** knows it can reach Router A at a cost of 1 and Router C at a cost of 1.
- **Router C** knows it can reach Router B at a cost of 1, but it cannot reach Router A (infinity).

**Routing Process**

1. **Router A sends its distance vector to Router B**:
   - B receives A's distance vector and updates its table.
2. **Router B updates its table**:
   - For Router C, B can calculate the cost to reach C through A:
     - Cost (A to C) = Cost (A to B) + Cost (B to C) = 1 + 1 = 2.
   - B updates its distance vector:

## 4) Explain Link state routing with suitable example.

# Ques-2

**Ch-5**

1) Demonstrate the various error detection techniques at data link layer with example.
2) Explain the following static channel allocations mechanisms: i) TDMA ii) FDMA
3) Explain p-persistent CSMA protocol in detail.
4) Data link protocols almost always put CRC in a trailer rather than in a header. Why?
5) State the difference between bit rate and baud rate.
6) Discuss the working of slotted aloha along with its efficiency in terms of channel utilization.
7) Explain different types of CSMA with suitable example.
8) Generate CRC code for message 1101010101. Given generator polynomial is x4+x2+1.